# In-gram: In-Context N-gram Memory for Hybrid Language Models with Fewer Attention Layers

Keston Aquino-Michaels
No Way Labs
https://github.com/no-way-labs/in-gram

**Abstract**

Recent open hybrid language models mix linear-attention and softmax-attention layers at ratios up to 7:1, and the softmax-attention cache grows with the context. One role of softmax attention is to recall earlier text verbatim. In software-agent trajectories, copying what followed the longest earlier match predicts 78.8% of next tokens. We introduce In-Context N-gram Memory (In-gram), in which each occurrence of an $N$-gram writes a note into a slot addressed by the $N$-gram's hash, and later occurrences read that slot at a constant cost per token. At equal training steps on 16k-token agentic contexts, In-gram lets a hybrid reach lower loss with fewer softmax-attention layers than a 3:1 hybrid without it. After 164M training tokens, an In-gram model with no softmax-attention layers is 0.102 nats per token below a parameter-matched 3:1 hybrid. With eight times more training (1.31B tokens), a 7:1 hybrid with In-gram is 0.06 nats below both a 3:1 and a 7:1 hybrid without it. In-gram's state in our models is 2.8 times smaller than the cache of a single softmax-attention layer after 16k tokens of a session, and 6.0 times smaller after 131k.

## 1 Introduction

The cache of a softmax-attention layer stores a key and a value for every token of the context [26] and therefore grows linearly with context length. For a model shaped like Kimi K3.1 Pro, we estimate that the attention cache and index of a single 1M-token session occupy 32.3 GB of accelerator memory (Section 5). Sparse attention reduces the portion of the cache that is read at each decoding step and leaves its size unchanged [23]. Recent open hybrid models reduce the cache size by mixing softmax-attention layers with a larger number of linear-attention layers, whose state has a fixed size. Qwen3.8-Flash-Next retains softmax attention in 12 of its 48 layers and uses Gated DeltaNet in the remainder [24, 25, 31]. Kimi K3.1 Pro has 69 layers of Kimi Delta Attention and 24 of softmax attention [15, 21], and Ling 3.1 Flash uses a 7:1 ratio [14]. GLM-5.3-Flash is the first model of the GLM series to combine linear and sparse attention, with three linear-attention layers for each sparse-attention layer [32]. The softmax-attention layers of Qwen3.8-Flash-Next and Kimi K3.1 Pro are likewise sparse as released, and MiniMax-M3.1-Flash uses sparse attention in every layer, with no linear-attention layer [20].

One role of the remaining softmax-attention layers is the retrieval of earlier tokens. The Qwen3.8-Flash-Next report states that these layers "retain direct token-level retrieval that is difficult for any finite-state recurrent memory to reproduce exactly" [24]. The GLM-5.3-Flash announcement assigns "local dependencies" to linear attention and the retrieval of "relevant global context" to sparse attention [32]. Agent trajectories frequently repeat their own earlier text: in SWE-rebench trajectories of 16,384 tokens, a lookup that copies the continuation of the longest earlier match predicts 78.8% of next tokens (Section 4.1).

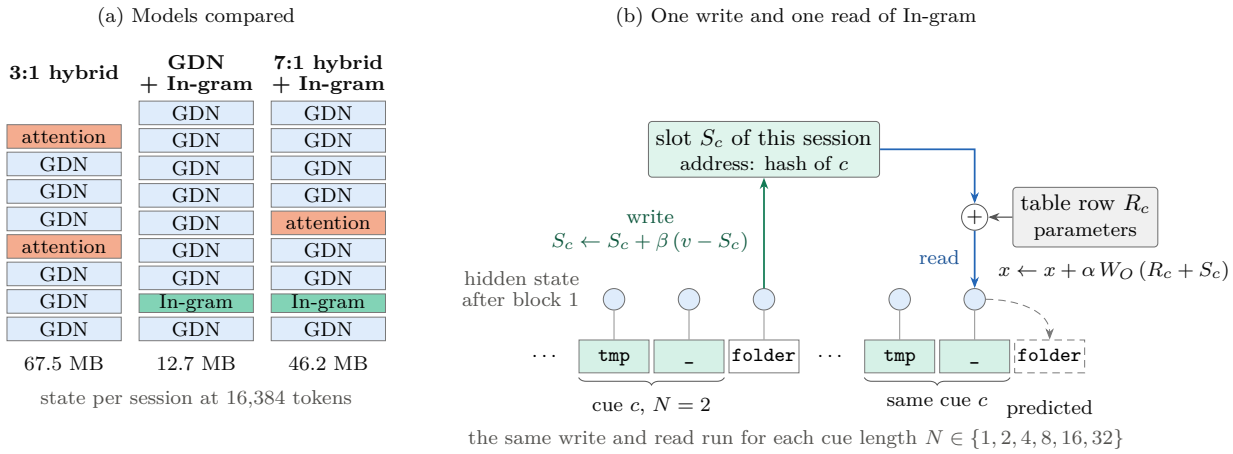



**Figure 1:** (a) The three models of our main comparisons, each with 8 blocks: a 3:1 hybrid with softmax attention in blocks 4 and 8, a Gated DeltaNet (GDN) model with In-gram after block 1, and a 7:1 hybrid with In-gram. The number below each model is the state that a decoder stores per session at 16,384 tokens (Section 5). (b) In-gram for one cue length, on tokens of a held-out trajectory that names a variable `tmp_folder`. When a token follows the cue $c$, here the two tokens `tmp` and `_`, the model computes a note $v$ from the hidden state at that token and writes it into the slot $S_c$, which is addressed by the hash of $c$. When $c$ ends at a later position, the model reads $S_c$, adds the learned table row $R_c$, and adds the result to the hidden state through a learned gate $\alpha$ and projection $W_O$; $\beta$ is a learned write strength. In the training trajectories this cue is most often followed by `path` and not by `folder` (Section 2.2).

Two other mechanisms use the statistics of earlier text without softmax attention, static hashed $N$-gram tables and cache language models. Static hashed $N$-gram tables, as in Engram [7] and the $N$-gram embedding of Qwen3.8-Flash-Next, are parameters: they encode statistics of the training corpus and contain no information from the current context. Cache language models [12, 16] interpolate the prediction of a model with a distribution over the tokens that occurred earlier in the context. As a baseline we use a cache of this kind that predicts the next token from counts of what followed earlier occurrences of the current $N$-gram, and we refer to it as the n-gram cache; it operates at inference time, outside the trained model, and has no learned memory. It shows that a lookup by exact $N$-gram match supplies recall that the softmax-attention layers of a hybrid have not learned: after 164M training tokens, a 3:1 hybrid of 30.4M parameters reaches 0.749 nats per token, and interpolating its predictions with those of the n-gram cache reduces this to 0.627 (Section 4.2). The cache, however, is not trained and changes only the output distribution; no layer of the model can read it, and the model keeps its softmax-attention layers and their cache.

We introduce In-Context N-gram Memory (In-gram; Figure 1), which performs this lookup within the model and is trained end to end. At each position, and for each of six cue lengths $N$ from 1 to 32, the model writes a note into a slot addressed by the hash of the preceding $N$ tokens. When the same $N$-gram ends at a later position, the model reads that slot. Because the addresses depend only on token ids, the schedule of writes and reads is known before the forward pass. Training is therefore parallel over the sequence, and decoding requires one read and one write per cue length for each token, independent of context length. In-gram adopts the hashed addressing of Engram and adds in-context writes. It is a write-based form of $N$-gram heads [1], which match earlier positions through an attention mask. We insert it once, after the first block of a Gated DeltaNet model or hybrid.

We evaluate In-gram in Gated DeltaNet models and hybrids of 55.9M to 115.3M parameters and compare them with hybrids of 30.4M to 88.4M parameters that lack it, all trained on SWE-rebench

agent trajectories with a context of 16,384 tokens. We compare models at equal training steps on eight seeds and commit each test to the repository before the runs that it adds (Section 3). After 164M training tokens, a model with In-gram and no softmax attention reaches a loss 0.102 nats per token lower than a parameter-matched 3:1 hybrid (Figure 3). A difference remains when the hybrid is given the same hashed table without writes (0.137 nats) and when the n-gram cache is interpolated into both models (0.023 nats). After 1.31B training tokens, a 7:1 hybrid with In-gram reaches 0.448 nats per token, compared with 0.510 for the 3:1 hybrid and 0.508 for the 7:1 hybrid without In-gram (Figure 4). In a session of 131,072 tokens, In-gram's state in our models is 6.0 times smaller than the cache of one softmax-attention layer (Figure 7). Section 5 estimates the consequence for serving: if a model shaped like Qwen3.8-Flash-Next maintains its loss with half of its softmax-attention layers and In-gram, the largest context that fits on a laptop with 128 GB of memory increases from 434k to 744k tokens.

## 2 Method

In-gram is a layer inserted between two blocks of a sequence model, and each insertion is a *placement*. At every position the layer writes to and reads from a memory whose entries are addressed by a hash of the preceding $N$ token ids, the *cue*; positions interact only through this memory. Figure 1b shows one write and one read for one cue length.

### 2.1 Cues, slots and table rows

Let $w_1, \dots, w_L$ be the token ids of a context, and let $x_t \in \mathbb{R}^d$ be the hidden state at position $t$ where the layer is inserted. For a cue length $N$, the cue at position $t \geq N$ is the $N$-gram $c = (w_{t-N+1}, \dots, w_t)$ that ends there. A polynomial hash maps it to a 61-bit address,

$$a_N(c) = \Big( \sum_{i=0}^{N-1} (w_{t-i} + 1)\, p_N^i \bmod (2^{61} - 1) \Big) \oplus s_N, \tag{1}$$

with a prime $p_N$ and a constant $s_N$ for each cue length. We use six cue lengths, $N \in \{1, 2, 4, 8, 16, 32\}$, and each has its own memory.

A cue addresses two vectors of width $d_m = 64$, a slot and a table row. The *slot* $S_c$ belongs to the current context: it is created at zero when the context first writes to $c$ and is keyed by the full address, and two different cues therefore share a slot only if their 61-bit addresses collide. The *table row* $R_c$ is a parameter that all contexts share, one row of a learned table with $2^{16}$ rows per cue length, selected by a second hash of the address (the splitmix64 finalizer) reduced modulo $2^{16}$. Different cues can share a row, as in hashed $N$-gram embeddings [7].

### 2.2 Write and read

At every position $p$ and for each cue length, the layer writes to the slot of the cue $c$ that ends at $p - 1$. The note $v_p$ and the write strength $\beta_p$ are computed from the hidden state at $p$:

$$v_p = W_v x_p, \qquad \beta_p = \sigma(u^\top x_p + b), \qquad S_c \leftarrow S_c + \beta_p\,(v_p - S_c). \tag{2}$$

$W_v \in \mathbb{R}^{d_m \times d}$, $u \in \mathbb{R}^d$ and $b$ are learned for each cue length. The note describes the token that followed the cue, and the slot is a weighted sum of the notes written under the cue: its weights sum to less than one because the slot starts at zero, and of two notes with equal write strength the later one has the larger weight.

At position $t$ and for each cue length, the layer reads the slot and the table row of the cue $c$ that ends at $t$:

$$r_t^N = R_c + S_c. \tag{3}$$

Here $S_c$ contains the writes of all positions before $t$. If the context has not yet written to $c$, $S_c$ is zero and a flag $e_t^N$ is set to 1. A position $t < N$ has no cue of length $N$; its read is zero with the flag set. Let $m_t$ be the concatenation of $r_t^N$ and $e_t^N$ over the six cue lengths. The layer adds a gated projection of $m_t$ to the hidden state:

$$\alpha_t = \sigma\Big(\frac{\langle \mathrm{norm}(x_t),\, \mathrm{norm}(W_K m_t)\rangle}{\sqrt{d}}\Big), \qquad x_t \leftarrow x_t + \alpha_t\, W_O m_t, \tag{4}$$

where norm is RMS normalization with a learned gain and $W_K, W_O \in \mathbb{R}^{d\times 6(d_m+1)}$. The gate $\alpha_t$ is a scalar that compares the read with the hidden state, and the model can use it to attenuate a read that does not fit the current context.

When the cue that ends at $t$ also ended at an earlier position $t'$, its slot contains a note computed from $x_{t'+1}$, the hidden state at the token that followed. The read therefore supplies position $t$, which predicts $w_{t+1}$, with a representation of the earlier continuations of its last $N$ tokens, for six values of $N$ at once. The table row is trained over the whole corpus, and it is the entire read for a cue that the context has not written. The example of Figure 1b shows the two roles: in the first 16,384 tokens of that held-out trajectory, the cue `tmp_` occurs 18 times and is followed by `folder` each time, whereas in the first 16,384 tokens of 2,000 sampled training trajectories it occurs 385 times, followed by `path` in 230 of them (59.7%) and by `folder` in none. With the writes disabled the layer reduces to a static $N$-gram table, which we use as a control (Section 4.2).

### 2.3 Training and decoding

Because the addresses depend on token ids alone, the positions that write to each slot, and the state that each read receives, are known before the forward pass. For every batch we compute this schedule on the CPU: the writes are sorted by slot and position, and each read receives a pointer to the last write that precedes it. The write of Equation 2 is an affine map of the slot, $S \mapsto (1-\beta)S + \beta v$, and a composition of affine maps is affine. A parallel prefix scan within each slot [13] thus yields the state after every write, at a depth logarithmic in the largest number of writes to one slot. The scan computes the same states as sequential execution of the writes, and gradients reach the notes, the write strengths, the gate and the table rows.

During decoding, each new token hashes one cue per cue length, reads one slot and one table row, and writes one slot, at a cost that is independent of the length of the context. The state of a session is one vector of width $d_m$ for every cue it has written, and a token adds at most one slot per cue length; Section 5 reports the measured counts.

### 2.4 Configuration

Unless we state otherwise, a model with In-gram uses one configuration, which we fixed before the confirmatory runs that use it. Its cue lengths were selected on seeds 0 and 1 (Appendix A.5), after an earlier confirmatory comparison with cue lengths 1 to 5 (Appendix C). It has the six cue lengths above, $d_m = 64$, $2^{16}$ table rows per cue length, and one placement after the first block. The table has $6 \times 2^{16} \times 64 = 25.2$M parameters, and at model width 256 the projections of the layer have 0.3M. The dose grid of Section 4.4 also trains models with two placements, after blocks 1 and 5, and with four, after blocks 1, 3, 5 and 7. Each placement has its own slots and projections, and all placements of a model share one table. Appendix E reports a variant, the scored read, which

keeps the last $K$ notes of a slot separately, each with a key, and weights them and the table row by attention from the reading position.

## 3 Experimental setup

We train and evaluate on SWE-rebench agent trajectories that were recorded with OpenHands [2, 22, 27]. Each trajectory is rendered as text in message order: the message text, each tool call as JSON of its name and arguments, and each tool output verbatim. The system prompt, which is the same in every trajectory, is omitted. We tokenize with the GPT-2 byte-pair vocabulary and use the first 16,384 trajectories of the dataset. A hash of the repository name assigns each repository to training or evaluation, and no repository therefore occurs in both: 15,680 trajectories (1.65B tokens) form the training set and 704 trajectories (69.8M tokens) are held out. A training example is a window of 16,384 consecutive tokens from one trajectory. Trajectories are drawn with probability proportional to their length, and the start of the window is uniform.

Every model is a stack of blocks, each of which applies a sequence mixer and then an MLP with one hidden layer of twice the model width and GELU activations; both are preceded by RMS normalization and have a residual connection. The mixer is either Gated DeltaNet (GDN) [31], in the implementation of the flash-linear-attention library, or causal softmax attention with rotary position embeddings. Both use heads of width 64 and a causal depthwise convolution of kernel size 4 on queries, keys and values. The input embedding and the output layer are separate matrices. $A$ counts the softmax-attention layers of a model and $P$ the placements of In-gram. With 8 blocks, a 3:1 hybrid has attention in blocks 4 and 8 and a 7:1 hybrid in block 4; the deeper 3:1 hybrids have attention in every fourth block. The dose grid of Section 4.4 also has models with no attention and with attention in every second block. We use three sizes: 8 blocks of width 256, 12 blocks of width 384 and 16 blocks of width 512. Unless we state otherwise, a model has 8 blocks of width 256, and In-gram has the configuration of Section 2.4.

All models train with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$), a weight decay of 0.1 on weight matrices other than the embedding and the hashed table, and gradient clipping at norm 1. A step is one window, and 10,000 steps therefore amount to 164M tokens. The learning rate rises linearly over 200 steps and then follows a cosine to 1% of its peak at the end of the run. For each model of Table 13 we selected the peak among two or three values on seeds 0 and 1. The 7:1 hybrids and the remaining cells of the dose grid have no selection of their own and use $4 \times 10^{-3}$ without In-gram and $2 \times 10^{-3}$ with it (Appendix D). Training uses bfloat16 autocast, with the reads and writes of In-gram in 32-bit floats, on one rented RTX 4090 per run (Appendix G).

We evaluate every model on the same 32 held-out windows of 16,384 tokens. *Loss* is the mean negative log-likelihood per token on these windows, in nats. We selected learning rates and cue lengths on seeds 0 and 1 (Tables 13 and 12). Confirmatory comparisons use seeds 20–27, which were not used for selection: $n = 8$ runs per arm, paired by seed. A seed sets the initialization and the order of the training windows: step $k$ of seed $s$ draws its window from a generator seeded with $1000s + k$, two seeds that differ by one therefore train on the same windows at an offset of 1,000 steps, and both models of a pair train on the same windows. For each comparison we committed the arms, the metric and the test to the repository before the runs of its new arms started; in rows 1 to 4 and 7 of Figure 3, one of the two arms consists of runs from an earlier comparison. The test is a one-sided Wilcoxon signed-rank test on the paired differences, with a Holm correction inside each pre-registered family of two to four tests. With $n = 8$ the smallest attainable $p$ is $1/256 = 0.0039$; the Holm-adjusted values 0.008, 0.012 and 0.016 in our tables therefore all indicate that the eight differences share one sign. We write $\Delta$ for the loss of the model with In-gram minus

the loss of the reference, and report its mean with a 95% percentile-bootstrap interval over seeds (10,000 resamples). Results on seeds 0 and 1 are exploratory and are labelled as such. The code that produced these results, the configuration, log and per-token losses of every run, a reference implementation of In-gram and the pre-registration files with their commit times are available at `https://github.com/no-way-labs/in-gram`.

# 4 Results

## 4.1 Repeats in agentic sessions and the n-gram cache

In the 32 evaluation windows, the $N$-gram that precedes a token has already occurred in the same window for 94.6% of tokens at $N = 1$, 77.0% at $N = 4$ and 65.5% at $N = 8$. A lookup with no parameters, which finds the longest earlier match ($N = 8$ down to 1) and copies the token that followed its most recent occurrence, predicts 78.8% of next tokens. These windows are cut from longer sessions: the held-out trajectories have a median of 96k tokens, and repeats become more frequent and longer with depth (Figure 2). The preceding 8-gram has occurred earlier for 53.2% of tokens at positions 8k to 16k and for 90.2% beyond 128k, and the lookup predicts 73.5% and 91.6% of tokens at these depths. The contexts of 16,384 tokens that we train and evaluate on therefore contain fewer of the repeats that In-gram reads than a full session does.

Because a lookup predicts this share of tokens, we include a baseline that operates at inference time, the n-gram cache. At each position it finds the longest $N \leq 8$ whose $N$-gram occurred earlier and predicts from the counts of its earlier continuations. We mix it with a model using one weight per matched $N$, fit the weights on 16 windows and score the other 16, in both directions. On seeds 0 and 1 the cache lowers the loss of GDN from 1.002 to 0.683 and the loss of the 3:1 hybrid from 0.722 to 0.621. We therefore report each $\Delta$ twice: between the two models, and between the two models with the cache mixed into both.

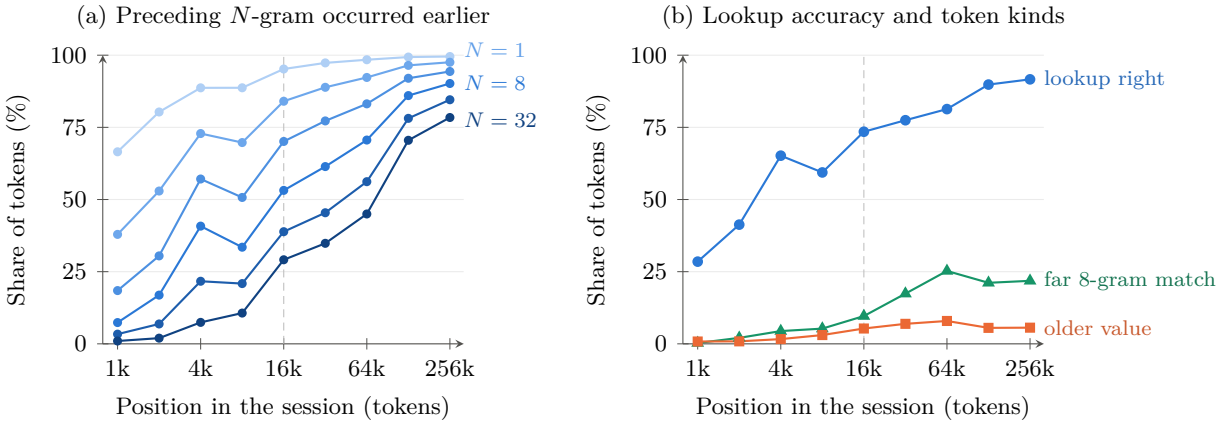



**Figure 2:** Repeat statistics of the 704 held-out trajectories, read from their first token, by position in the session. (a) Share of tokens whose preceding $N$-gram occurred earlier in the session, for the six cue lengths of In-gram. (b) Share of tokens that the longest-match lookup of Section 4.1 predicts, share whose preceding 8-gram occurred earlier with its most recent occurrence more than 512 tokens back, and share for which an older continuation of a 4-token cue is right and the most recent one is wrong. Each point pools one range of positions (up to 1k, 1k to 2k, 2k to 4k, and so on); 566 trajectories reach 64k tokens and 134 reach 128k. The dashed line marks 16,384 tokens, the context of our models. These are counts on token ids and involve no model.

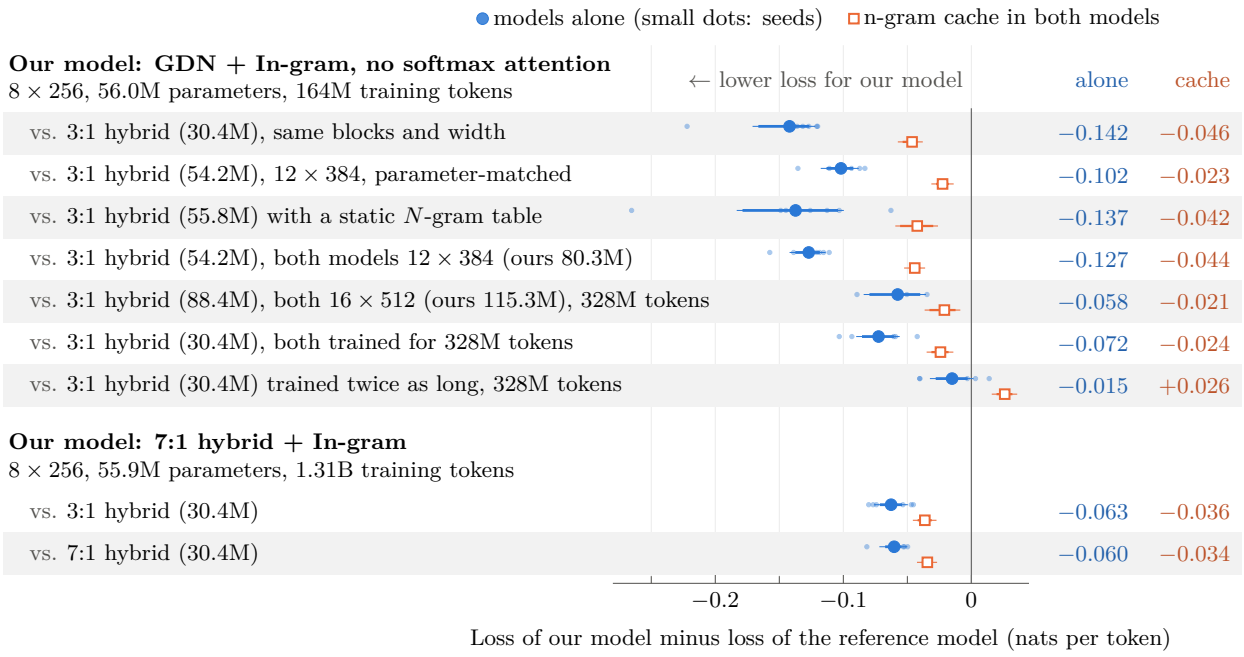



**Figure 3:** The pre-registered comparisons. Each bold heading names our model, and each row below it compares that model with one reference model that has no In-gram. The horizontal axis is the loss of our model minus the loss of the reference; left of zero, our model has the lower loss. Large markers are means over seeds 20–27 ($n = 8$; $n = 4$ in the fifth row) with 95% bootstrap intervals, small dots are single seeds, and open squares repeat the comparison with the n-gram cache interpolated into both models. The columns at the right give the two means. The line under each heading gives the blocks × width, parameters and training tokens of our model, and a row states where either model differs; parentheses after a reference give its parameters. Both models of a row train for the same number of steps, except in the seventh row, where the hybrid trains twice as long. Losses and tests are in Table 2.

## 4.2 Replacing the attention layers of a 3:1 hybrid

Figure 3 shows every pre-registered comparison, and Table 2 in the appendix gives the losses and tests in the same order. At equal training steps, a GDN with In-gram and no attention reaches a lower loss than each 3:1 hybrid we trained. Against the 3:1 hybrid of the same depth and width (30.4M parameters), the loss is 0.607 against 0.749, and $\Delta = -0.142$ nats (95% CI $[-0.166, -0.127]$; 8 of 8 seeds; Holm $p = 0.012$). One hybrid run is an outlier (seed 20, loss 0.835); the median difference is $-0.134$.

The model with In-gram has 56.0M parameters, of which 25.2M are rows of the hashed table; the 30.4M hybrid is therefore the smaller model. The second and third rows of Figure 3 are controls for that difference. A 3:1 hybrid with 12 blocks of width 384 has 54.2M parameters and reaches 0.709, and $\Delta = -0.102$ ($[-0.113, -0.092]$; 8 of 8; Holm $p = 0.008$). A 3:1 hybrid that includes the same hashed table with its writes disabled, a static $N$-gram table, has 55.8M parameters (0.1M of them are write projections that it does not use) and reaches 0.745, against 0.749 for the hybrid without the table, and $\Delta = -0.137$ ($[-0.179, -0.104]$; 8 of 8; Holm $p = 0.016$). Table parameters without the writes therefore do not account for the difference. The difference also holds at two larger sizes (fourth and fifth rows): $-0.127$ with 12 blocks of width 384 and $-0.058$ with 16 blocks of width 512, the latter on four seeds.

The open squares of Figure 3 interpolate the n-gram cache into both models. In these five rows, each $\Delta$ retains its sign and decreases in magnitude to between $-0.046$ and $-0.021$. The

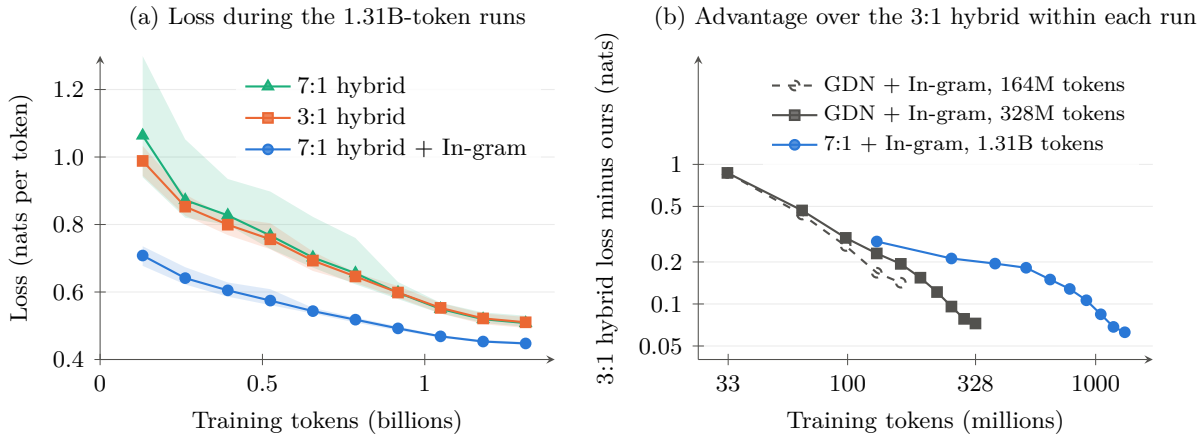



**Figure 4:** (a) Held-out loss during the 1.31B-token runs of the 7:1 hybrid with In-gram, the 3:1 hybrid and the 7:1 hybrid; lines are means over seeds 20–27 and bands span the lowest and highest seed. The final losses are 0.448, 0.510 and 0.508 (last two rows of Figure 3). (b) Loss of the 3:1 hybrid minus loss of the model with In-gram, at the evaluations within each group of runs (the legend gives our model and the training tokens of the group), on logarithmic axes. Every run follows a cosine schedule over its own length; only the last point of each curve is a finished run.

cache recovers 0.123 nats for the 30.4M hybrid (0.749 to 0.627) and 0.027 nats for the model with In-gram (0.607 to 0.580). Against the parameter-matched hybrid the remaining difference is $-0.023$ ($[-0.028, -0.018]$; 8 of 8; Holm $p = 0.008$).

### 4.3 Longer training, and a 7:1 hybrid with In-gram

The difference from the 3:1 hybrid decreases when both models are trained for longer (sixth row of Figure 3). With both at 20,000 steps (328M tokens), $\Delta = -0.072$ ($[-0.085, -0.061]$; 8 of 8; Holm $p = 0.016$), half of the $-0.142$ at 164M tokens. A further pre-registered test allows the hybrid twice the steps of our model (seventh row). It bounds a comparison at equal wall-clock, because our training steps with In-gram were 1.07 to 1.19 times slower (Appendix A.3). The model with In-gram retains the lower loss ($\Delta = -0.015$, $[-0.028, -0.003]$; 6 of 8 seeds; Holm $p = 0.039$). With the n-gram cache on both sides the hybrid has the lower loss ($+0.026$, $[+0.020, +0.032]$; 0 of 8 seeds).

The longest runs test a hybrid that retains one of the two attention layers (Figure 4a). We trained three models for 80,000 steps (1.31B tokens, less than one pass over the 1.65B tokens of the training set): a 7:1 hybrid with In-gram, the 3:1 hybrid and the 7:1 hybrid, all with the learning rates of the 164M-token runs (Section 3). The 7:1 hybrid with In-gram reaches 0.448, which is below the 3:1 hybrid at 0.510, the primary test ($\Delta = -0.063$, $[-0.071, -0.054]$; 8 of 8; Holm $p = 0.008$), and below the 7:1 hybrid at 0.508 ($\Delta = -0.060$, $[-0.067, -0.055]$; 8 of 8; Holm $p = 0.008$). With the n-gram cache on both sides the differences are $-0.036$ and $-0.034$, each on 8 of 8 seeds.

Without In-gram, the two hybrids end at the same loss: the 3:1 hybrid minus the 7:1 hybrid is $+0.002$ (95% CI $[-0.011, +0.014]$; $n = 8$; post hoc). At 1.31B tokens the 7:1 hybrid with In-gram is therefore 0.06 nats below a hybrid without it at either attention count. Within these runs, its advantage over the 3:1 hybrid decreases from 0.280 nats at 8,000 steps to 0.063 at 80,000, and by 0.006 over the last 8,000 steps, when the cosine schedule is near its floor (Figure 4b).

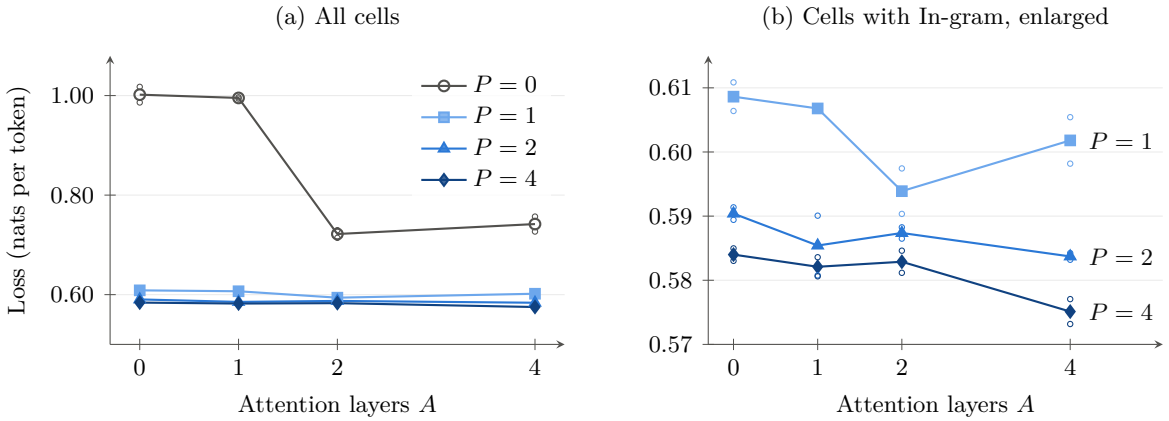



**Figure 5:** Loss against the number of attention layers $A$, for $P \in \{0, 1, 2, 4\}$ placements of In-gram (8 blocks of width 256, 164M training tokens). Without In-gram ($P = 0$) the loss falls from 1.002 to 0.722 between no attention and two layers; with one placement all four models lie between 0.594 and 0.609. Lines join seed means and small open circles are the two seeds. Panel (b) repeats the models with $P \geq 1$ on a finer scale. Seeds 0 and 1 only: the grid is exploratory. Seed means and per-seed values are in Tables 10 and 11.

### 4.4 Contribution of attention when In-gram is present

We trained every combination of $A \in \{0, 1, 2, 4\}$ attention layers and $P \in \{0, 1, 2, 4\}$ placements (Figure 5). Each cell has two seeds and trains for 164M tokens; the grid is exploratory, with its analyses fixed before it ran. Without In-gram, the loss depends on the attention layers: 1.002 with none, 0.995 with one, 0.722 with two and 0.742 with four. With one placement, all four cells lie between 0.594 and 0.609, and the largest difference between two attention counts is 0.015 nats. Further placements lower the loss at every $A$, to between 0.575 and 0.584 at $P = 4$. One placement without any attention layer is already 0.113 nats below the 3:1 hybrid without In-gram.

On the same grid, we decompose the cost of removing both attention layers, $C(P) = L(A{=}0, P) - L(A{=}2, P)$, by token type (Figure 6). Without In-gram it is 0.280 nats per token ($[0.215, 0.344]$). Among the types with at least 1% of tokens, it is largest on far exact repeats, the tokens that continue a match of at least 8 tokens whose previous occurrence lies more than 512 tokens back ($+0.646$), and on cues whose earlier continuations all agree ($+0.465$). With one placement the cost over all tokens is 0.015 ($[0.010, 0.020]$) and the cost on far exact repeats is 0.000 ($[-0.003, 0.003]$).

The remaining cost at $P = 1$ is concentrated on three types, with 2.1%, 5.4% and 4.7% of tokens. These are older-value tokens, where the matched cue had several earlier continuations and the most recent one is wrong ($+0.053$, $[0.036, 0.073]$); tokens with no earlier match ($+0.053$, $[0.026, 0.099]$); and format tokens ($+0.041$, $[0.013, 0.102]$). In the confirmatory runs (Table 5; post hoc), far exact repeats are 16.4% of tokens and likewise account for 60%, 76% and 57% of the difference from the 3:1 hybrid at 164M, 328M and 1.31B tokens. Without attention, the model with In-gram is worse than the 3:1 hybrid on older-value tokens ($+0.089$, $[0.042, 0.138]$ at 328M tokens). With one attention layer, at 1.31B tokens, this type no longer favours the 3:1 hybrid ($-0.013$, $[-0.027, 0.000]$).

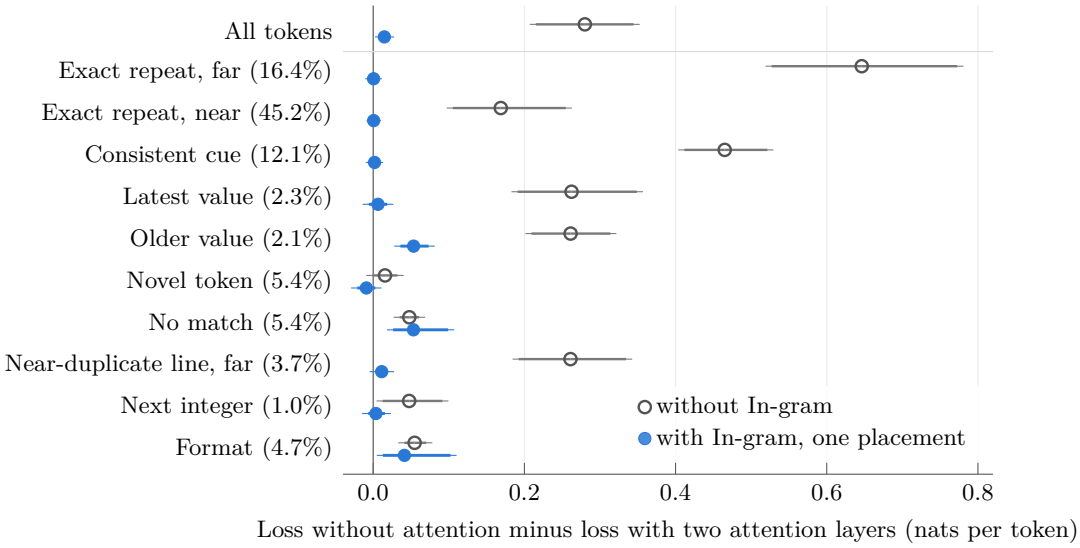



**Figure 6:** Cost of removing both attention layers of a 3:1 hybrid, by token type: the loss with no attention layer minus the loss with two, without In-gram (open circles) and with one placement (filled). Without In-gram the cost is largest on far exact repeats (0.646 nats); with one placement it is 0.000 there and 0.015 over all tokens. Bars are 95% bootstrap intervals over the 32 evaluation windows, with seeds 0 and 1 averaged (exploratory). Percentages are shares of evaluation tokens; types overlap and are defined in Table 4. Table 6 gives all placements and the one type with under 0.5% of tokens.

# 5 Serving implications

In-gram changes the state that a served model stores for each session, and where that state can reside. We first report measured counts and then present two scenarios from a roofline model. The scenarios assume a model with fewer attention layers; Section 4 provides the evidence, at up to 115M parameters, that In-gram allows a hybrid to maintain its loss with fewer of them.

## 5.1 State per session

A model with In-gram has two components beyond its other weights, a hashed table and a slot store. The hashed table is a parameter: all sessions share it, and each token reads one row per cue length. The slot store belongs to one session: it contains one vector for every cue the session has written, located through a hash index. Each generated token reads and writes one slot per cue length, regardless of the length of the context. An attention layer stores a key and a value for every earlier token, and each generated token scores all of them, or an index over them when the attention is sparse. The slot store therefore requires little memory bandwidth and can reside in ordinary system memory or on storage, whereas the attention cache requires fast memory. The schedule of writes and reads depends on token ids alone, and prefill therefore fills the slots in one pass over the prompt.

The slot store grows with the context, because a session adds a slot each time it encounters a new cue, at most six per token. On held-out trajectories read from their first token, the measured rate is below this bound and decreases with depth (Figure 7a). A session has written 2.89 slots per token after 16,384 tokens, 1.96 after 65,536 tokens and 1.34 after 131,072 tokens, and between 65,536 and 131,072 tokens a new token adds 0.67 slots. After 131,072 tokens the store therefore holds 1.34 vectors of width 64 per token of context, against a key and a value of width 256 in one

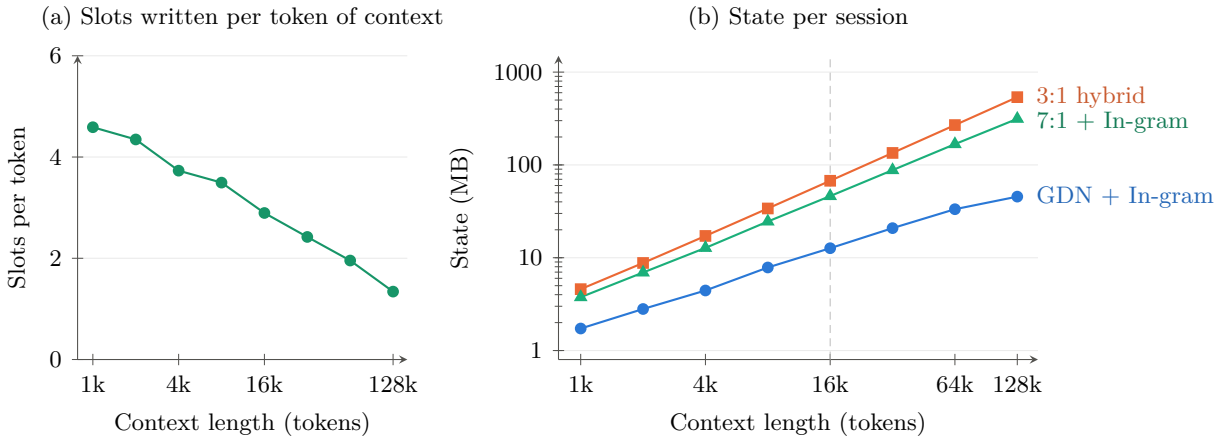



**Figure 7:** (a) Slots that In-gram has written per token of context, for held-out trajectories read from their first token: 704 trajectories up to 16,384 tokens, 566 at 65,536 tokens and 134 at 131,072 tokens. The bound is six. (b) State per session of our three models with 8 blocks of width 256, in fp32, computed from these slot counts with the accounting of Table 3. At 131,072 tokens it is 537 MB for the 3:1 hybrid, 314 MB for the 7:1 hybrid with In-gram and 46 MB for GDN with In-gram. The dashed line marks 16,384 tokens, the context at which the models were trained and evaluated.

attention layer, a factor of 6.0; after 16,384 tokens the factor is 2.8. With the state of the GDN blocks included, a session of 16,384 tokens occupies 12.7 MB for GDN with In-gram, 46.2 MB for the 7:1 hybrid with In-gram and 67.5 MB for the 3:1 hybrid in fp32 (Appendix A.3), and a session of 131,072 tokens 46 MB, 314 MB and 537 MB (Figure 7b). Slot vectors and table rows quantize to 4 bits at a cost of at most 0.0005 nats on seeds 0 and 1 (exploratory; Appendix E).

## 5.2 Two scenarios

For one user on one machine, memory determines how much context fits (Table 1a). As released, the 12 attention layers of a model shaped like Qwen3.8-Flash-Next require 25.3 KB of cache and index per token of context. With six attention layers the cache requires 12.7 KB per token, and In-gram adds 1.6 KB per token and a table of 0.4 GB. On a laptop with 128 GB of unified memory, of which the model may use 110 GB and the weights and $N$-gram tables occupy 99 GB, the largest context that fits increases from 434k tokens to 744k with six attention layers and to 1.06M with four. Both scenarios assume 1.5 slots per token, above the 1.34 that we measure after 131,072 tokens.

The same accounting applies to a replica that serves many sessions (Table 1b). For a model shaped like Kimi K3.1 Pro, a session of 1M tokens occupies 32.7 GB on the accelerators, 32.3 GB of it attention cache and index. With half of the attention layers a session occupies 16.7 GB there and 3.3 GB of In-gram slots in host memory, and a replica accommodates 149 sessions instead of 76, or 289 with a quarter of the attention layers. A decode step takes 12 to 14 ms in all three configurations at this context, because reading the weights dominates it, and tokens per second therefore scale with the number of sessions.

(a) One user: largest context that fits in memory, in tokens

| Machine | 12 attention layers (as released) | 6 attention layers + In-gram | 4 attention layers + In-gram |
|---|---|---|---|
| RTX 4090, 24 GB + 128 GB RAM | 674k | 1.35M | 2.02M |
| Laptop, 128 GB unified memory | 434k | 744k | 1.06M |
| Mini PC, 128 GB unified memory | 631k | 1.10M | 1.56M |

(b) Shared replica: state per session and sessions per replica

| | 128k-token sessions | | | 1M-token sessions | | |
|---|---|---|---|---|---|---|
| Configuration | GPU state (GB) | host state (GB) | sessions per replica | GPU state (GB) | host state (GB) | sessions per replica |
| 24 attention layers (as released) | 4.5 | 0.0 | 558 | 32.7 | 0.0 | 76 |
| 12 attention layers + In-gram | 2.5 | 0.4 | 988 | 16.7 | 3.3 | 149 |
| 6 attention layers + In-gram | 1.6 | 0.4 | 1,605 | 8.6 | 3.3 | 289 |

**Table 1:** Two scenarios from a roofline model. (a) A model shaped like Qwen3.8-Flash-Next (48 layers, 12 of them sparse attention as released) serving one user: weights and N-gram tables at 4.5 bits per parameter, attention cache in 16-bit floats, In-gram with slot width 512 in 16-bit floats, 1.5 slots per token and 24 bytes of index per slot. The model may use 110 GB of the laptop's memory and 115 GB of the mini PC's; the laptop has 546 GB/s of memory bandwidth and the mini PC 273 GB/s. On the first machine the cache and the weights outside the experts are in 22 GB of the GPU memory, and the experts, the tables and the slot store are in 120 GB of system memory. (b) A model shaped like Kimi K3.1 Pro (69 linear and 24 sparse-attention layers as released) on a replica of 32 accelerators with 141 GB of memory each, 90% of it usable: weights of 1,561 GB as released (4.5 bits per parameter on average), attention cache in 16-bit floats, sessions limited by the accelerator memory left after the weights, removed attention layers become linear layers, and In-gram uses slot width 1,024 in 16-bit floats with 1.5 slots per token and 24 bytes of index per slot, stored in host memory.

# 6 Related work

Hybrid models interleave softmax attention with linear-attention or state-space layers [15, 23, 24], and the releases cited in Section 1 retain different numbers of softmax layers. One reported cost of replacing softmax layers is long-context retrieval: in an ablation of the GLM-5 report, a 1:1 hybrid with Gated DeltaNet, adapted from a 9B model with attention in every layer, scores 76.76 on RULER at 64k tokens, against 85.35 for that model [10]. Sparse attention reduces what a decoding step reads from the cache, and MATCH [18] retrieves chunks of the input with an external retriever and unmasks their positions in sliding-window attention layers. In-gram addresses the number of softmax-attention layers and leaves the remaining layers unchanged.

Engram [7] adds hashed $N$-gram embedding tables to a Transformer as a static memory, and Qwen3.8-Flash-Next has 51B parameters of such tables [24]. These tables are parameters; In-gram uses the same addressing and adds slots that the current context writes. Delta N-gram [29] writes a sparse patch over the hashed $N$-gram table of a pretrained model with a block-averaged delta rule at inference time. Its writes persist across requests and its writer is trained separately on the frozen model, whereas the slots of In-gram are empty at the start of every context and are trained with the model. $N$-gram heads [1] retrieve by exact match of the preceding $N$ token ids within the model: an attention mask selects the positions that follow earlier occurrences, and their hidden states are averaged uniformly. In-gram replaces the mask of that implementation by hashed slots, at a constant cost per token in training and decoding, and the uniform average by a learned write strength, a gate and a table row; the authors note that a trie could serve the

matches at inference. Transformers trained on Markov chains learn induction heads that implement smoothed in-context $N$-gram estimators [9]. At the output of a model, cache language models [12, 16] and pointer mixtures [19] interpolate the predicted distribution with a distribution over earlier tokens; our n-gram cache baseline belongs to this group. Copy-Paste language models [11] train an attention-based copy head beside the output head, and report that a sliding-window model with this head matches a full-attention model at 12 times less memory per token. The copy head stores one vector per token of context and changes the predicted distribution. In-gram stores one vector per distinct cue and changes the hidden state, which the later layers read.

Other work extends the state of recurrent models with sparsely addressed memory. Sparse Delta Memory [6] replaces the dense key-value outer product of Gated DeltaNet by sparse reads and writes to a large explicit memory with product-key addresses, and learns the initial state of that memory as a parametric component. RAM-Net [30] selects slots of a fixed array with a learned address decoder. Latest exact match attention [5] replaces softmax attention heads: queries and keys are binarized, and each query attends to the latest key identical to it, at a cost per token that is independent of the context length. HOLA [8] adds to a delta-rule state a bounded cache of the exact keys and values of the tokens with the largest write residual, Resona [28] retrieves chunks of the input context with a context encoder and a query encoder for linear recurrent models, and Titans [3] update a neural memory at test time. Product-key memories [4, 17] are trained key-value tables without in-context writes. These methods compute their keys or addresses with the model, whereas In-gram derives its addresses from token ids alone: the schedule of writes and reads is fixed before the forward pass (Section 2.3), and two cues that differ in one token address unrelated slots. We regard the two kinds of memory as compatible: in the hybrids of Cabannes et al. [6], Sparse Delta Memory raises long-context retrieval above Gated DeltaNet while full attention retains the higher average accuracy on RULER, and the authors note that the two may provide complementary capabilities. A learned sparse memory enlarges the state for approximate matches, and In-gram adds an index for exact ones.

# 7 Discussion

Softmax attention and In-gram perform the same kind of operation, in that each adds to the hidden state at a position a vector retrieved from earlier positions of the context; they differ in the key used for the retrieval. Attention computes its query and keys from hidden states and stores a key and a value for every token, whereas the key of In-gram is the identity of the last $N$ tokens, which is known before the forward pass, and In-gram stores one vector for every distinct cue. Our results indicate how much of the retrieval in agentic contexts a key of this kind covers. With one placement, the cost of removing both attention layers of a 3:1 hybrid decreases from 0.280 to 0.015 nats per token (Figure 6), and a model with no attention layer reaches a lower loss than the 3:1 hybrid at equal training steps (Figure 3). The remaining cost falls on tokens whose continuation an exact match of token ids does not determine: cues with several earlier continuations of which the most recent is wrong, tokens with no earlier match, and format tokens.

We conjecture that the attention layers of a hybrid learn a lookup of this kind during training, and that In-gram provides it from the first step. Two observations are consistent with this conjecture: the difference between GDN with In-gram and the 3:1 hybrid decreases by half between 164M and 328M training tokens, with far exact repeats accounting for more than half of it (Section 4.4), and with In-gram present the attention layers place less of their far attention on the positions that follow an earlier occurrence of the current bigram (Appendix A.4). The conjecture does not extend to older-value tokens, on which the model without attention is worse than the hybrid. Our runs also

leave open whether the difference vanishes with further training: at 1.31B tokens it is 0.063 nats for the 7:1 hybrid with In-gram, and it decreased by 0.006 over the last 8,000 steps.

The first open question is retrieval at long context: our models are trained and evaluated at 16,384 tokens, and long-context retrieval is the capability that the 1:1 hybrids of the GLM-5 ablation lose [10]; benchmarks of it would test whether a hybrid with fewer attention layers and In-gram retains it. The second is scale: our largest model has 115.3M parameters and our longest run 1.31B training tokens. The third is the comparison with other in-context memories, specifically $N$-gram heads, memories with learned addresses such as Sparse Delta Memory and latest exact match attention, and a trained copy head at the output [11]. For devices with a fixed memory budget, we also plan to evaluate a bounded slot store and to measure decoding speed with the slots in system memory.

## References


[1] Ekin Akyürek, Bailin Wang, Yoon Kim, and Jacob Andreas. In-context language learning: Architectures and algorithms. In *International Conference on Machine Learning*, pages 787–812, 2024.

[2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2025.

[3] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. In *Advances in Neural Information Processing Systems*, 2025.

[4] Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen-tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. In *International Conference on Machine Learning*, pages 3831–3842, 2025.

[5] Moritz Brösamle. Latest exact match attention. *arXiv preprint arXiv:2609.25802*, 2026.

[6] Loïc Cabannes, Pierre-Emmanuel Mazaré, Ilze Amanda Auzina, Gergely Szilvasy, Maria Lomeli, Matthijs Douze, Justin Carpentier, Gabriel Synnaeve, and Hervé Jégou. Sparse Delta Memory: Scaling the state of linear RNNs through sparsity. In *Advances in Neural Information Processing Systems*, 2026.

[7] Xin Cheng, Rui Tian, Wangding Zeng, Damai Dai, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026. The version in the Proceedings of ACL 2026, pages 4968–4990, names the module Deep Sparse Embedding.

[8] Wanyun Cui. A hippocampus for linear attention: An exact memory for what the recurrent state forgets. *arXiv preprint arXiv:2607.02303*, 2026.

[9] Francesco D'Angelo, Oğuz Kaan Yüksel, Swathi Shree Narashiman, and Nicolas Flammarion. Induction heads interpolate N-grams. In *International Conference on Machine Learning*, 2026.

[10] GLM-5 Team. GLM-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

[11] Nathan Godey and Yoav Artzi. Copy-Paste language models. *arXiv preprint arXiv:2610.19247*, 2026.

[12] Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

[13] W. Daniel Hillis and Guy L. Steele, Jr. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.

[14] inclusionAI. Ling-3.1-flash. Model card, 2026. URL `https://huggingface.co/inclusionAI/Ling-3.1-flash`.

[15] Kimi Team, Yu Zhang, Zongyu Lin, et al. Kimi Linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.

[16] Roland Kuhn and Renato De Mori. A cache-based natural language model for speech recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583, 1990.

[17] Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, 2019.

[18] Linrui Ma, Chun Hei Lo, Xinyu Wang, et al. MATCH: Modulating attention via in-context retrieval for long-context transformers. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pages 15165–15179, 2026.

[19] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

[20] MiniMax. MiniMax-M3.1-Flash. Model card, 2026. URL `https://huggingface.co/MiniMaxAI/MiniMax-M3.1-Flash`.

[21] Moonshot AI. Kimi K3.1 Pro. Model card, 2026. URL `https://huggingface.co/moonshotai/Kimi-K3.1-Pro`.

[22] Nebius. SWE-rebench OpenHands trajectories. Dataset, 2025. URL `https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories`.

[23] NVIDIA. Nemotron 3.5 Super and Lightning: Open, efficient mixture-of-experts hybrid Mamba-Transformer models for agentic AI. Technical report, NVIDIA, 2026. URL `https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3.5-Super-and-Lightning-Technical-Report.pdf`.

[24] Zihan Qiu, Zekun Wang, Xiao Li, et al. On the design of Qwen3.8-Next architecture: Evaluation, efficiency, and training stability. *arXiv preprint arXiv:2608.30320*, 2026.

[25] Qwen. Qwen3.8-Flash-Next. Model card, 2026. URL `https://huggingface.co/Qwen/Qwen3.8-Flash-Next`.

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[27] Xingyao Wang, Boxuan Li, Yufan Song, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025.

[28] Xinyu Wang, Linrui Ma, Jerry Huang, Peng Lu, Prasanna Parthasarathi, Xiao-Wen Chang, Boxing Chen, and Yufei Cui. Resona: Improving context copying in linear recurrence models with retrieval. In *Conference on Language Modeling*, 2025.

[29] Hao Wu. Delta N-gram: Forward-only continual writing into a language model's native N-gram memory. *arXiv preprint arXiv:2610.31125*, 2026.

[30] Kaicheng Xiao, Haotian Li, Liran Dong, and Guoliang Xing. RAM-Net: Linear-time sequence modeling with sparsely addressable state. In *Advances in Neural Information Processing Systems*, 2026.

[31] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated Delta Networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

[32] Z.ai. GLM-5.3-Flash. Blog post, 2026. URL `https://z.ai/blog/glm-5.3-flash`.

# A Tests and additional results

## A.1 Losses and tests of the pre-registered comparisons

| Ours | Reference | Training tokens | Loss ours | Loss ref. | $\Delta$ | $\Delta < 0$ | $p_{\text{Holm}}$ | $\Delta$, cache |
|---|---|---|---|---|---|---|---|---|
| *GDN with In-gram and no softmax attention, against 3:1 hybrids* | | | | | | | | |
| $8 \times 256$ (56.0M) | $8 \times 256$ (30.4M) | 164M | 0.607 | 0.749 | $-0.142$ | 8/8 | 0.012 | $-0.046$ |
| | | | | | $[-0.166, -0.127]$ | | | $[-0.054, -0.042]$ |
| $8 \times 256$ (56.0M) | $12 \times 384$ (54.2M) | 164M | 0.607 | 0.709 | $-0.102$ | 8/8 | 0.008 | $-0.023$ |
| | | | | | $[-0.113, -0.092]$ | | | $[-0.028, -0.018]$ |
| $8 \times 256$ (56.0M) | $8 \times 256$ + table (55.8M) | 164M | 0.607 | 0.745 | $-0.137$ | 8/8 | 0.016 | $-0.042$ |
| | | | | | $[-0.179, -0.104]$ | | | $[-0.056, -0.030]$ |
| $12 \times 384$ (80.3M) | $12 \times 384$ (54.2M) | 164M | 0.582 | 0.709 | $-0.127$ | 8/8 | 0.016 | $-0.044$ |
| | | | | | $[-0.137, -0.118]$ | | | $[-0.049, -0.040]$ |
| $16 \times 512$ (115.3M) | $16 \times 512$ (88.4M) | 328M | 0.503 | 0.561 | $-0.058$ | 4/4 | n/a | $-0.021$ |
| | | | | | $[-0.080, -0.040]$ | | | $[-0.033, -0.012]$ |
| $8 \times 256$ (56.0M) | $8 \times 256$ (30.4M) | 328M | 0.550 | 0.622 | $-0.072$ | 8/8 | 0.016 | $-0.024$ |
| | | | | | $[-0.085, -0.061]$ | | | $[-0.031, -0.018]$ |
| $8 \times 256$ (56.0M) | $8 \times 256$ (30.4M) | 164M/328M | 0.607 | 0.622 | $-0.015$ | 6/8 | 0.039 | $+0.026$ |
| | | | | | $[-0.028, -0.003]$ | | | $[+0.020, +0.032]$ |
| *7:1 hybrid with In-gram, against hybrids without it* | | | | | | | | |
| $8 \times 256$ (55.9M) | 3:1, $8 \times 256$ (30.4M) | 1.31B | 0.448 | 0.510 | $-0.063$ | 8/8 | 0.008 | $-0.036$ |
| | | | | | $[-0.071, -0.054]$ | | | $[-0.042, -0.031]$ |
| $8 \times 256$ (55.9M) | 7:1, $8 \times 256$ (30.4M) | 1.31B | 0.448 | 0.508 | $-0.060$ | 8/8 | 0.008 | $-0.034$ |
| | | | | | $[-0.067, -0.055]$ | | | $[-0.039, -0.031]$ |

**Table 2:** Losses and tests for the comparisons of Figure 3, in the same order, at 16k context on seeds 20–27 ($n = 8$, paired; $n = 4$ at $16 \times 512$). Models are given as blocks $\times$ width with their parameter count; in the first group every reference is a 3:1 hybrid, and "+ table" marks the one with a static $N$-gram table. Loss is the mean over seeds; $\Delta$ is ours minus reference, with its 95% bootstrap interval over seeds below it. $p_{\text{Holm}}$ is the one-sided Wilcoxon $p$ after Holm correction within the pre-registered family. In the row marked 164M/328M our model trains for 164M tokens and the reference for 328M (Section 4.3). The with-cache difference was tested in the first two rows (same adjusted $p$) and is reported without a test in the others. Per-seed losses are in Tables 8 and 9.

Table 2 lists the losses and tests behind Figure 3. Across the 32 evaluation windows, the share of tokens that the lookup of Section 4.1 predicts ranges from 63.4% to 98.4%, and from 8k tokens of depth on, tokens for which an older continuation is the right one stay between 5.3% and 7.9% (Figure 2b). In the first row, the mean difference without the outlier seed is $-0.131$. The hybrid of the parameter-matched control has 54.2M parameters, of which 15.6M lie outside the embedding and output layers, against 5.1M for our model outside embedding, output and table. With 12 blocks of width 384, GDN with In-gram has 80.3M parameters against 54.2M for the 3:1 hybrid, and with 16 blocks of width 512, trained for 20,000 steps on four seeds per arm, 115.3M against 88.4M.

## A.2 Training length

Between 164M and 328M training tokens the hybrid improves by 0.127 nats and the model with In-gram by 0.057. On seeds 0 and 1 at 164M tokens, the 7:1 hybrid reaches 0.995 without In-gram and 0.607 with it, against 0.722 for the 3:1 hybrid (Figure 5); the 1.31B-token runs test whether this ordering persists with eight times more training. Without In-gram, the benefit of the second attention layer decreases with training length (post hoc). It is 0.27 nats at 164M tokens on seeds 0

and 1, and 0.075 nats at 8,000 steps of the long runs. From 56,000 steps on, the two means differ by at most 0.003 (Figure 4a), and at the end 3 of 8 seeds have the 3:1 hybrid below the 7:1 hybrid. The model with In-gram also varies less across seeds at the end of training: the standard deviation over seeds is 0.002, against 0.013 for the 3:1 hybrid and 0.010 for the 7:1 hybrid.

### A.3 Step time and state

| | | | Seconds per step | | Peak GPU | State at 16k |
|---|---|---|---|---|---|---|
| Model | Steps | Parameters | median | range | memory (GB) | tokens (MB) |
| 3:1 hybrid, $8 \times 256$ | 10k | 30.4M | 0.122 | 0.101–0.125 | 4.3 | 67.5 |
| GDN + In-gram, $8 \times 256$ | 10k | 56.0M | 0.130 | 0.116–0.198 | 5.8 | 12.7 |
| 3:1 hybrid + static table, $8 \times 256$ | 10k | 55.8M | 0.138 | 0.123–0.148 | 5.7 | 67.5 |
| 3:1 hybrid, $12 \times 384$ | 10k | 54.2M | 0.156 | 0.150–0.175 | 6.6 | 151.9 |
| GDN + In-gram, $12 \times 384$ | 10k | 80.3M | 0.167 | 0.161–0.185 | 8.2 | 13.3 |
| 3:1 hybrid, $8 \times 256$ | 80k | 30.4M | 0.122 | 0.099–0.135 | 4.3 | 67.5 |
| 7:1 hybrid, $8 \times 256$ | 80k | 30.4M | 0.122 | 0.093–0.127 | 4.4 | 34.0 |
| 7:1 hybrid + In-gram, $8 \times 256$ | 80k | 55.9M | 0.131 | 0.118–0.160 | 5.8 | 46.2 |

**Table 3:** Cost of the models in Table 2. Seconds per step are wall-clock time per training step, including data loading and the periodic evaluations, over seeds 20–27; at each evaluation a model with In-gram runs three additional passes with its reads ablated, which are included. Each run used one rented RTX 4090, and hosts differ between runs. State is the fp32 size of what a decoder stores per session after 16,384 tokens, computed as described in Appendix A.3; Figure 7 extends it to longer contexts.

Training steps with In-gram were slower in our runs (Table 3). Over seeds 20–27, GDN with In-gram takes a median of 0.130 seconds per step (range 0.116 to 0.198) and the 3:1 hybrid 0.122 (0.101 to 0.125). Because the runs used different rented hosts, the times are unpaired: the ratio of medians is 1.07 and the ratio of means is 1.19. The test of Section 4.3 that allows the hybrid twice the steps favours the hybrid for any step-time ratio below 2.

We compute the state that a decoder stores per session at 16,384 tokens, in fp32 (Table 3): keys and values of the model width for every token in each attention layer, the recurrent state of each GDN block, and one 64-dimensional vector per written slot of In-gram (47,426 slots on average for a held-out session read from its first token; the hash index is not counted). This amounts to 12.7 MB for GDN with In-gram, 67.5 MB for the 3:1 hybrid and 46.2 MB for the 7:1 hybrid with In-gram. The evaluation windows are cut inside sessions and contain fewer new cues: 29,337 slots on average, or 1.79 per token.

### A.4 Dose grid and attention statistics

In the grid of Figure 5, four attention layers reach 0.732 at their better learning rate, and a model of eight attention layers with no GDN reaches 0.766 at its better learning rate (Table 13). On these two seeds the 3:1 hybrid is therefore the best model without In-gram. With two placements the four cells lie between 0.584 and 0.590.

The summary we fixed in advance is the iso-loss attention $A^*(P)$: the number of attention layers that a model with $P$ placements needs in order to match the 3:1 hybrid without In-gram (0.722), found by linear interpolation over $A$. For $P = 1$, 2 and 4, the cell with no attention is already below that loss on both seeds, by 0.113, 0.131 and 0.138 nats in the seed mean, and hence $A^*(P) \leq 0$ for every $P$ we ran.

With four placements the cost of removing both attention layers is 0.001 over all tokens ($[-0.001, 0.003]$), and the intervals for older-value tokens ($+0.016$, $[0.003, 0.029]$) and for tokens with no match ($+0.013$, $[0.005, 0.021]$) still exclude zero (Table 6). In-gram also covers near-duplicate lines, which we had expected to depend on attention: their cost decreases from $+0.261$ at $P = 0$ to $+0.011$ at $P = 1$. Tokens with no match change least ($+0.048$ and $+0.053$). On seeds 20–27 without attention, the model with In-gram is also worse than the 3:1 hybrid on format tokens ($+0.012$, $[0.005, 0.020]$ at 328M tokens); with one attention layer, at 1.31B tokens, it is not ($-0.007$, $[-0.013, -0.004]$).

To measure where the attention layers attend, we define the induction share of far attention as the fraction of the attention mass on keys more than 512 tokens back that falls on positions directly after an earlier occurrence of the current bigram, averaged over heads and layers (Table 7). In the 3:1 cells it is 0.233 without In-gram and 0.136, 0.181 and 0.174 at $P = 1$, 2 and 4; with four attention layers it is 0.211 and then 0.154, 0.150 and 0.109. The share is lower with In-gram in the seed mean of all six cells and in ten of the twelve single-seed comparisons.

### A.5 Cue lengths and further checks

On seeds 0 and 1, longer cues lower the loss (Table 12): 0.633 with cue lengths 1 to 5, 0.614 with $\{1, 2, 4, 8, 16\}$ at the same parameter count, and 0.606 with $\{1, 2, 4, 8, 16, 32\}$. We selected between $\{1, 2, 3, 5, 8, 16\}$ and $\{1, 2, 4, 8, 16, 32\}$ on these seeds by a rule fixed in advance, the lower mean loss. On seeds 20–27, the selected set reaches 0.607 against 0.632 for cue lengths 1 to 5, $\Delta = -0.025$ ($[-0.028, -0.022]$; 8 of 8; Holm $p = 0.012$); this was the primary test of its family. The n-gram cache contributes less when the cues are long: it lowers the loss by 0.038 nats with cue lengths 1 to 5 and by 0.027 with the selected set, and with the cache on both sides the difference between the two cue sets is $-0.014$ ($[-0.016, -0.012]$; reported, not tested). Placements and long cues combine: on seeds 0 and 1, two placements reach 0.608 with cue lengths 1 to 5 and 0.590 with the selected set (Table 10).

Appendix E reports three further checks: a scored read rule lowers the loss by 0.005 nats in a smaller setting and was worse than the additive read on both selection seeds of this recipe; rounding the rows of the hashed table and the slot state at each read to 4 bits changes the loss by at most 0.0005 nats; and we detect no increase in looping under generation at 32 prompts and two seeds.

### A.6 Slot growth and the serving scenarios

The slot rate of Section 5.1 varies between sessions: after 16,384 tokens it ranges from 1.19 to 4.06 slots per token across the 704 held-out trajectories, and after 131,072 tokens from 0.55 to 2.02 across the 134 trajectories of that length. One held-out trajectory reaches 262,144 tokens; at positions between 128k and 256k, a new token adds 0.55 slots on average over the trajectories that reach them. Long cues account for most of the slots: after 16,384 tokens, cue length 32 has written 0.82 slots per token and cue length 1 has written 0.10. Our decoder stores the hash index on the host and the slot vectors on the accelerator, and all three of our model sizes use slot width 64. Our implementation also has a bounded store, in which a cue hashes to a set of four slots and a new cue evicts the least recently used slot of its set; a device with a fixed memory budget requires such a store, and we plan to evaluate it on this recipe.

In the scenario of Table 1a, each attention layer requires 2.1 KB per token and In-gram 1.6 KB per token in total; the added context comes from the attention caches that are removed. At the bound of six slots per token, the largest contexts on the laptop are 559k tokens with six attention layers and 719k with four. Decode speed is unchanged in the roofline model (142 to 146 tokens

per second at 262,144 tokens of context on the laptop), because sparse attention reads a bounded number of tokens per step. In a decode benchmark of an earlier configuration, In-gram added 1.3 to 1.4 ms per token to GDN, and neither model slowed down between 4,096 and 1M tokens of context (Appendix F).

# B Token types

| Token type | A target token has this type when |
|---|---|
| Exact repeat, far | the longest earlier match of its preceding context has at least 8 tokens, the token after the most recent occurrence of that match is the target, and that occurrence lies more than 512 tokens back |
| Exact repeat, near | the same, with the occurrence at most 512 tokens back |
| Consistent cue | the longest match has 2 to 7 tokens and every earlier continuation of it equals the target |
| Latest value | the longest match has 2 to 7 tokens and at least two distinct earlier continuations, and the most recent one is the target |
| Older value | as for latest value, but the target is an earlier continuation other than the most recent one |
| Novel token | the target token has not appeared earlier in the window |
| No match | not even the one token before the target has an earlier occurrence |
| Near-duplicate line, far | there is no exact match of 8 tokens, the current line has at least 4 distinct token bigrams, and it shares at least half of them (Jaccard) with a line that ends at least 512 tokens before it and contains the target |
| Closing bracket, far | the target closes a bracket that was opened more than 64 tokens back |
| Next integer | the target is an integer $n$ and the nearest earlier integer within 200 tokens is $n-1$ |
| Format | there is no exact match of 8 tokens, the target is punctuation or whitespace, and the most recent earlier position preceded by the same sequence of token classes (the 5 to 12 tokens before the target in its line) continued with the same token |

**Table 4:** Token types used in Figure 6 and Tables 6 and 5. Types are computed from the tokens of each evaluation window alone and can overlap; their shares therefore do not sum to 100%. Match lengths are measured on the ladder 1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48 and 64, and a match of 2 to 7 tokens is therefore evaluated at length 2, 3, 4 or 6. We fixed the definitions before the dose grid ran.

| Token type | % | GDN + In-gram 164M tokens | GDN + In-gram 328M tokens | 7:1 hybrid + In-gram 1.31B tokens |
|---|---|---|---|---|
| All tokens | 100.0 | −0.142 [−0.187, −0.102] | −0.072 [−0.102, −0.046] | −0.063 [−0.084, −0.044] |
| Exact repeat, far | 16.4 | −0.515 [−0.642, −0.401] | −0.337 [−0.429, −0.252] | −0.216 [−0.290, −0.152] |
| Exact repeat, near | 45.2 | −0.045 [−0.070, −0.027] | −0.016 [−0.026, −0.009] | −0.009 [−0.014, −0.005] |
| Consistent cue | 12.1 | −0.202 [−0.261, −0.156] | −0.111 [−0.151, −0.078] | −0.064 [−0.090, −0.044] |
| Latest value | 2.3 | −0.104 [−0.133, −0.075] | −0.037 [−0.059, −0.017] | −0.038 [−0.049, −0.027] |
| Older value | 2.1 | +0.018 [−0.026, +0.060] | +0.089 [+0.042, +0.138] | −0.013 [−0.027, +0.000] |
| Novel token | 5.4 | −0.216 [−0.283, −0.173] | −0.163 [−0.239, −0.117] | −0.112 [−0.155, −0.079] |
| No match | 5.4 | −0.110 [−0.166, −0.073] | −0.063 [−0.117, −0.029] | −0.063 [−0.088, −0.043] |
| Near-duplicate line, far | 3.7 | −0.117 [−0.196, −0.060] | −0.040 [−0.082, −0.007] | −0.051 [−0.088, −0.023] |
| Closing bracket, far | 0.1 | −0.199 [−0.448, −0.001] | −0.116 [−0.302, +0.029] | −0.178 [−0.295, −0.095] |
| Next integer | 1.0 | −0.094 [−0.188, −0.028] | −0.057 [−0.112, −0.018] | −0.055 [−0.126, −0.010] |
| Format | 4.7 | +0.016 [+0.004, +0.038] | +0.012 [+0.005, +0.020] | −0.007 [−0.013, −0.004] |

**Table 5:** Loss of the model with In-gram minus loss of the 3:1 hybrid, by token type, for the three pairs of Table 2 at 8 blocks of width 256 that train both arms equally long (seeds 20–27 averaged per token). The second column is the share of evaluation tokens. Brackets are 95% bootstrap intervals over the 32 evaluation windows (10,000 resamples). Exploratory; the claims are in Section 4.4.

| Token type | % | $P = 0$ | $P = 1$ | $P = 2$ | $P = 4$ |
|---|---|---|---|---|---|
| All tokens | 100.0 | +0.280 [+0.215, +0.344] | +0.015 [+0.010, +0.020] | +0.003 [+0.001, +0.005] | +0.001 [−0.001, +0.003] |
| Exact repeat, far | 16.4 | +0.646 [+0.527, +0.773] | +0.000 [−0.003, +0.003] | −0.003 [−0.006, +0.000] | −0.001 [−0.002, +0.001] |
| Exact repeat, near | 45.2 | +0.169 [+0.105, +0.255] | +0.001 [−0.001, +0.002] | −0.001 [−0.002, +0.001] | −0.001 [−0.004, +0.000] |
| Consistent cue | 12.1 | +0.465 [+0.412, +0.521] | +0.002 [−0.002, +0.005] | −0.001 [−0.004, +0.002] | −0.001 [−0.004, +0.001] |
| Latest value | 2.3 | +0.262 [+0.191, +0.349] | +0.006 [−0.006, +0.018] | −0.004 [−0.011, +0.003] | −0.005 [−0.015, +0.005] |
| Older value | 2.1 | +0.261 [+0.209, +0.314] | +0.053 [+0.036, +0.073] | +0.012 [−0.004, +0.025] | +0.016 [+0.003, +0.029] |
| Novel token | 5.4 | +0.016 [−0.001, +0.032] | −0.009 [−0.021, +0.003] | +0.008 [−0.001, +0.019] | −0.007 [−0.017, +0.002] |
| No match | 5.4 | +0.048 [+0.035, +0.061] | +0.053 [+0.026, +0.099] | +0.019 [+0.004, +0.040] | +0.013 [+0.005, +0.021] |
| Near-duplicate line, far | 3.7 | +0.261 [+0.193, +0.335] | +0.011 [+0.003, +0.020] | +0.001 [−0.007, +0.009] | +0.003 [−0.001, +0.008] |
| Closing bracket, far | 0.1 | +0.628 [+0.327, +0.963] | +0.074 [−0.088, +0.273] | −0.034 [−0.123, +0.055] | +0.117 [−0.048, +0.288] |
| Next integer | 1.0 | +0.048 [+0.012, +0.091] | +0.004 [−0.007, +0.016] | −0.002 [−0.013, +0.008] | −0.008 [−0.014, −0.002] |
| Format | 4.7 | +0.055 [+0.041, +0.070] | +0.041 [+0.013, +0.102] | +0.012 [+0.000, +0.036] | +0.005 [−0.002, +0.013] |

**Table 6:** Cost of removing both attention layers, $C(P) = L(A{=}0, P) - L(A{=}2, P)$ in nats per token, by token type and number of placements $P$ (Figure 6 plots $P = 0$ and $P = 1$). The second column is the share of evaluation tokens of each type. Brackets are 95% bootstrap intervals over the 32 evaluation windows (10,000 resamples, windows resampled jointly for both models, seeds 0 and 1 averaged). Exploratory; the claims are in Section 4.4.

| | | Induction share of far attention | | | Far attention |
|---|---|---|---|---|---|
| | | seed 0 | seed 1 | mean | mass (mean) |
| $A = 2$ | $P = 0$ | 0.235 | 0.231 | 0.233 | 0.588 |
| $A = 2$ | $P = 1$ | 0.135 | 0.136 | 0.136 | 0.483 |
| $A = 2$ | $P = 2$ | 0.157 | 0.206 | 0.181 | 0.471 |
| $A = 2$ | $P = 4$ | 0.238 | 0.110 | 0.174 | 0.530 |
| $A = 4$ | $P = 0$ | 0.245 | 0.178 | 0.211 | 0.500 |
| $A = 4$ | $P = 1$ | 0.104 | 0.205 | 0.154 | 0.445 |
| $A = 4$ | $P = 2$ | 0.160 | 0.139 | 0.150 | 0.392 |
| $A = 4$ | $P = 4$ | 0.072 | 0.146 | 0.109 | 0.420 |

**Table 7:** Induction share of far attention on the dose grid, as defined in Appendix A.4, and the mean attention mass on keys more than 512 tokens back. Seeds 0 and 1; exploratory.

# C Per-seed losses

No run was dropped from any comparison; a run that failed on the platform was relaunched with the same seed. Tables 8 and 9 list every confirmatory run and Table 11 both seeds of every cell of the dose grid.

The first column of Table 8 is the model with cue lengths 1 to 5. Before we adopted the longer cues, a pre-registered single test had compared it with the 3:1 hybrid on the same seeds: $\Delta = -0.117$ (95% CI $[-0.143, -0.099]$; 8 of 8 seeds; $p = 0.0039$).

| Seed | GDN + In-gram, $8 \times 256$ $N \leq 5$ | GDN + In-gram, $8 \times 256$ $N \leq 32$ | 3:1 hybrid $8 \times 256$ | 3:1 hybrid $12 \times 384$ | 3:1 + static table, $8 \times 256$ | GDN + In-gram $12 \times 384$ |
|---|---|---|---|---|---|---|
| 20 | 0.6348 | 0.6129 | 0.8350 | 0.7238 | 0.7618 | 0.5850 |
| 21 | 0.6296 | 0.6048 | 0.7409 | 0.7403 | 0.8702 | 0.5829 |
| 22 | 0.6307 | 0.6079 | 0.7484 | 0.7093 | 0.7112 | 0.5836 |
| 23 | 0.6360 | 0.6009 | 0.7216 | 0.7022 | 0.7458 | 0.5837 |
| 24 | 0.6327 | 0.6036 | 0.7237 | 0.6972 | 0.7295 | 0.5818 |
| 25 | 0.6287 | 0.6088 | 0.7404 | 0.6919 | 0.7435 | 0.5809 |
| 26 | 0.6326 | 0.6101 | 0.7371 | 0.6973 | 0.6729 | 0.5764 |
| 27 | 0.6312 | 0.6089 | 0.7473 | 0.7108 | 0.7216 | 0.5825 |
| Mean | 0.6320 | 0.6073 | 0.7493 | 0.7091 | 0.7446 | 0.5821 |
| With cache | 0.5942 | 0.5804 | 0.6267 | 0.6029 | 0.6227 | 0.5587 |

**Table 8:** Loss per seed at 164M training tokens (10,000 steps), for the comparisons of Table 2 and Appendix A.5. The last row is the mean over seeds after mixing each model with the n-gram cache.

| | 328M tokens, $8 \times 256$ | | 328M tokens, $16 \times 512$ | | 1.31B tokens, $8 \times 256$ | | |
|---|---|---|---|---|---|---|---|
| Seed | GDN + In-gram | 3:1 | GDN + In-gram | 3:1 | 7:1 + In-gram | 3:1 | 7:1 |
| 20 | 0.5505 | 0.6097 | 0.5040 | 0.5386 | 0.4478 | 0.5220 | 0.4976 |
| 21 | 0.5496 | 0.6225 | 0.5013 | 0.5572 | 0.4461 | 0.5263 | 0.5090 |
| 22 | 0.5517 | 0.5941 | 0.5030 | 0.5533 | 0.4482 | 0.4935 | 0.5298 |
| 23 | 0.5480 | 0.6414 | 0.5050 | 0.5943 | 0.4482 | 0.5250 | 0.5113 |
| 24 | 0.5482 | 0.6217 | | | 0.4521 | 0.5119 | 0.5053 |
| 25 | 0.5458 | 0.6490 | | | 0.4460 | 0.4995 | 0.5082 |
| 26 | 0.5535 | 0.6282 | | | 0.4442 | 0.5092 | 0.4965 |
| 27 | 0.5513 | 0.6121 | | | 0.4480 | 0.4950 | 0.5053 |
| Mean | 0.5498 | 0.6223 | 0.5033 | 0.5609 | 0.4476 | 0.5103 | 0.5079 |
| With cache | 0.5302 | 0.5544 | 0.4879 | 0.5090 | 0.4375 | 0.4738 | 0.4718 |

**Table 9:** Loss per seed for the longer runs of Table 2. The third size ran on seeds 20–23 only.

| Attention layers | $P = 0$ | $P = 1$ | $P = 2$ | $P = 4$ |
|---|---|---|---|---|
| $A = 0$ (GDN) | 1.002 | 0.609 | 0.590 | 0.584 |
| $A = 1$ (7:1) | 0.995 | 0.607 | 0.585 | 0.582 |
| $A = 2$ (3:1) | 0.722 | 0.594 | 0.587 | 0.583 |
| $A = 4$ (1:1) | 0.742 | 0.602 | 0.584 | 0.575 |

**Table 10:** Seed-mean loss on the dose grid of Figure 5 (seeds 0 and 1). Cells with $P = 0$ train at learning rate $4 \times 10^{-3}$ and cells with $P \geq 1$ at $2 \times 10^{-3}$ (Table 13).

| Attention layers | $P = 0$ | $P = 1$ | $P = 2$ | $P = 4$ |
|---|---|---|---|---|
| $A = 0$ | 1.0175 / 0.9861 | 0.6109 / 0.6064 | 0.5914 / 0.5894 | 0.5850 / 0.5830 |
| $A = 1$ | 0.9965 / 0.9940 | 0.6068 / 0.6068 | 0.5901 / 0.5808 | 0.5806 / 0.5836 |
| $A = 2$ | 0.7141 / 0.7295 | 0.5974 / 0.5903 | 0.5883 / 0.5865 | 0.5846 / 0.5812 |
| $A = 4$ | 0.7568 / 0.7268 | 0.6054 / 0.5982 | 0.5832 / 0.5842 | 0.5771 / 0.5732 |

**Table 11:** Loss on the dose grid, seed 0 / seed 1 (means in Table 10).

| Cue lengths $N$ | Parameters | Seed 0 | Seed 1 | Mean | Mean with cache |
|---|---|---|---|---|---|
| 1, 2, 3, 4, 5 | 51.7M | 0.6349 | 0.6316 | 0.633 | 0.595 |
| 1, 2, 3, 4, 5, 8 | 56.0M | 0.6237 | 0.6217 | 0.623 | 0.592 |
| 1, 2, 3, 5, 8 | 51.7M | 0.6243 | 0.6244 | 0.624 | 0.593 |
| 1, 2, 4, 8, 16 | 51.7M | 0.6161 | 0.6124 | 0.614 | 0.586 |
| 1, 2, 3, 5, 8, 16 | 56.0M | 0.6111 | 0.6112 | 0.611 | 0.583 |
| 1, 2, 4, 8, 16, 32 | 56.0M | 0.6102 | 0.6020 | 0.606 | 0.579 |

**Table 12:** Loss by cue set for GDN with In-gram ($P = 1$, 8 blocks of width 256, 164M tokens), seeds 0 and 1 (exploratory; used to select the cue set, Appendix A.5). Each cue length adds one table of $2^{16}$ rows of width 64. The last column interpolates the model with the n-gram cache.

# D Learning-rate checks

For each model of Table 13 we compared two or three learning rates on the selection seeds, and each confirmatory arm in the table uses the rate with the lowest mean. The 7:1 hybrid and the 7:1 hybrid with In-gram have no comparison of their own: they use $4 \times 10^{-3}$, the rate of the 3:1 hybrid, and $2 \times 10^{-3}$, the rate of GDN with In-gram, as do the remaining cells of the dose grid without and with In-gram. In-gram's rate of $2 \times 10^{-3}$ was tuned with cue lengths 1 to 5 and fixed before the runs with long cues; the rows for long cues are a sensitivity check and selected nothing. At 16 blocks of width 512 each rate has one seed. The 80,000-step runs of Section 4.3 reuse the rates selected at 10,000 steps.

| Model | Learning rate | Seed 0 | Seed 1 | Mean |
|---|---|---|---|---|
| GDN, $8 \times 256$ | 4e-3 | 1.0175 | 0.9861 | 1.002 |
| | 8e-3 | 1.1702 | 1.0317 | 1.101 |
| 3:1 hybrid, $8 \times 256$ | 2e-3 | 0.7329 | 0.7175 | 0.725 |
| | 4e-3 | 0.7141 | 0.7295 | 0.722 |
| 1:1 hybrid, $8 \times 256$ | 2e-3 | 0.7439 | 0.7202 | 0.732 |
| | 4e-3 | 0.7568 | 0.7268 | 0.742 |
| Attention only, $8 \times 256$ | 2e-3 | 0.7905 | 0.7408 | 0.766 |
| | 4e-3 | 0.8148 | 0.7847 | 0.800 |
| GDN + In-gram, $8 \times 256$ | 1e-3 | 0.6239 | 0.6218 | 0.623 |
| | 2e-3 | 0.6102 | 0.6020 | 0.606 |
| | 4e-3 | n/a | 0.6106 | n/a |
| 3:1 hybrid + static table, $8 \times 256$ | 2e-3 | 0.7293 | 0.7554 | 0.742 |
| | 4e-3 | 0.7425 | 0.7333 | 0.738 |
| 3:1 hybrid, $12 \times 384$ | 2e-3 | 0.6772 | 0.7481 | 0.713 |
| | 4e-3 | 0.7139 | 0.7767 | 0.745 |
| GDN + In-gram (cues 1–5), $12 \times 384$ | 1e-3 | 0.6127 | 0.6175 | 0.615 |
| | 2e-3 | 0.6078 | 0.6093 | 0.609 |
| 3:1 hybrid, $16 \times 512$ | 1e-3 | 0.5692 | n/a | n/a |
| | 2e-3 | 0.5540 | n/a | n/a |
| GDN + In-gram, $16 \times 512$ | 1e-3 | 0.5038 | n/a | n/a |
| | 2e-3 | 0.5017 | n/a | n/a |

**Table 13:** Loss by learning rate on the selection seeds, at 164M training tokens (328M for 16 blocks of width 512), 16k context. An entry of n/a marks a run that was not made or was lost to a host failure.

# E Read rule, quantization and generation

All results above use the additive read; in a smaller setting (4 blocks of width 256, 4,096-token context, the first 4,096 trajectories, 4 windows per step, 128 evaluation windows, cue lengths 1 to 5, seeds 13–20), we pre-registered a single test of a scored read, which stores $K$ notes per slot and weights them by a key computed from the context. With $K = 4$ it lowers the loss from 0.829 to 0.824, $\Delta = -0.005$ ($[-0.008, -0.003]$; 8 of 8 seeds; $p = 0.0039$), and stores 8 vectors of state per slot where the additive read stores one. The difference with $K = 8$ is $-0.012$ ($[-0.015, -0.009]$; 8 of 8; reported, not tested). On the recipe of this paper (long cues, 16k context), the scored read with $K = 8$ was worse than the additive read on both selection seeds (0.629 and 0.608 against 0.610 and 0.602; one learning rate).

We also checked quantization on checkpoints from seeds 0 and 1 that use cue lengths 1 to 5 (exploratory). Under round-to-nearest quantization applied at evaluation, 4-bit rows of the hashed table change the loss of GDN with In-gram by +0.0001, and rounding the slot state to 4 bits at each read by +0.0005; the writes are computed at full precision. With 4-bit weights in the linear layers whose input width is a multiple of 64, and 8-bit weights in the embedding and the output layer, that model loses +0.003 and +0.002 on the two seeds, against +0.007 and +0.008 for GDN and +0.010 and +0.009 for the 3:1 hybrid; the read projections $W_K$ and $W_O$ of In-gram, of input width 325 in these checkpoints, remain at full precision. Keys and values of the hybrid at 8 bits cost at most +0.001. Our 4-bit keys and values are quantized per token, without per-channel scaling, and cost +0.350 and +0.064 on the two seeds.

For generation we provided the same checkpoints with 32 prompts of 4,096 tokens and sampled 256 new tokens. We classify a continuation as a loop when its last 128 tokens are periodic with a period of at most 64. The reference continuations loop in 7 of the 32 prompts. At temperature 1, every model loops in 7 to 9. Under greedy decoding the counts are 16 to 21 for the models with

In-gram, 16 and 19 for the 3:1 hybrid, and 18 and 20 for GDN. We detect no increase in looping with In-gram at 32 prompts and two seeds. Greedy continuations of the models with In-gram share more 8-grams with the prompt (0.69 to 0.70) than those of the hybrid (0.58 and 0.64) or the references (0.62).

# F Decode benchmark

We timed token-by-token decoding once, on an RTX 4090 at batch size 1, for untrained models of 4 blocks and width 256 on tokens of WikiText-103: 256 decoded tokens after a prefill of 4,096 to 1,048,576 tokens. The In-gram model in this benchmark is an earlier configuration with ten slot streams (cue lengths 1 to 5 under two write rules) of slot width 32, placed on the embeddings, with a separate static table of $2^{17}$ rows per stream that has its own gate in place of the additive table row. Per decoded token after 4,096 and 1,048,576 tokens of context, GDN took 4.91 and 4.90 ms, GDN with In-gram 6.22 and 6.26 ms, a hybrid with one attention layer 4.78 and 5.79 ms, and a model of four attention layers 4.06 and 5.97 ms. In-gram thus adds 1.3 to 1.4 ms per token at both lengths; the benchmark records one total per model and does not separate the host-side hashing from the reads and writes on the accelerator.

# G Compute

Every training run reported in this paper used one rented NVIDIA RTX 4090 with 24 GB of memory. The machines came from the Vast.ai marketplace and were rented through Rhombic (`https://rhombic.space`), an orchestration service that launched the runs, tracked their cost and collected their logs. On hosts of different speed, a run of 10,000 steps took 17 to 33 minutes, a run of 20,000 steps 39 to 74 minutes, and a run of 80,000 steps 2.1 to 3.6 hours. The 88 runs behind Figure 3 took 103 GPU-hours of training and were billed at 64 US dollars with their relaunches, and a run of 80,000 steps cost 1.49 dollars on average. All 534 logged runs of the project, which include the selection runs and exploratory runs that this paper does not report, took 319 hours of training time. The account that rented the machines was billed 213 dollars in total, for 700 rentals and 420 hours at 0.51 dollars per hour on average, which is an upper bound on the cost of the project. Data preparation, the token statistics of Section 4.1 and all analyses ran on CPUs.

# H Use of AI tools

The author proposed the method, the research questions and the framing of this paper, decided which experiments to run, reviewed the results and the text, and is responsible for the content of the paper. An AI coding assistant (Claude, Anthropic, used through Claude Code) wrote most of the code under the author's direction, launched and monitored the training runs, wrote the analysis scripts, searched and summarized the related work, and drafted the text and the figures. Three practices limit the errors that this division of work can introduce. Each confirmatory comparison was committed to the repository as a pre-registration before its runs started. The numbers in the results are produced by scripts from the run logs and are not typed by hand. The descriptions of the most closely related papers were checked against the full text of each paper.